Linguagem de Programação
Exercícios Ponteiros

1) Considerando as definições em C apresentadas a seguir, responda às questões formuladas.

```
float valores[10], *p;
p = &valores[0];
printf("%p %p\n", valores, p);
p = valores;
printf("%p %p\n", valores, p);
p = valores + 1;
// %x número inteiro em formato hexadecimal
printf("%x %p\n", p - valores, p);
p = valores + 6;
printf("%x %p\n", p - valores, p);
(*p) = 15;
```

a) O que significa mencionar, no código de um programa, apenas o nome de um vetor já declarado?

R.: Em C, mencionar apenas o nome de um vetor (como valores) equivale a usar um ponteiro constante que aponta para o primeiro elemento do vetor (&valores[0]). Ele representa o endereço de memória inicial da estrutura.

b) O que acontece se, ao final das instruções anteriores, fizermos (*p) = 15 ?

*R.: Como p recebeu valores + 6 nas instruções anteriores, ele aponta para o 7º elemento do vetor (valores[6]). A instrução (*p) = 15; vai atribuir o valor 15 à posição valores[6] do vetor.*

c) Qual o significado do resultado impresso no primeiro valor do último printf() do código?

R.: O trecho p - valores realiza uma aritmética de ponteiros. Como p aponta para valores + 6 e valores aponta para o início (+ 0), a subtração entre dois ponteiros do mesmo tipo retorna o número de elementos de distância entre eles. Portanto, o valor impresso será 6 (ou 6 em formato hexadecimal, que continua sendo 6).

2) [Questão 38 do Poscomp 2007 - Alterada] Dado o seguinte programa escrito em C:

```
#include <stdio.h>  int
main(void)
{ int n[] = {70, 80, 90};
  int *p;
  p = &n[0];
  p++;
  printf("Valor: %d ", *p);
  (*p)++;
  printf("Valor: %d\n", *p);
}
```

Qual é a resposta que será impressa na tela:

a) Valor: 70 Valor : 80

b) Valor: 70 Valor: 70

c) Valor: 80 Valor: 90

d) Valor: 80 Valor: 81

e) Valor: 90 Valor: 90.

R.: Inicialmente, p aponta para n[0] (cujo valor é 70).

A operação p++ faz o ponteiro avançar para a próxima posição de memória do tipo int, passando a apontar para n[1] (cujo valor é 80). O primeiro printf imprime: Valor: 80.

*A operação ((*p)++) incrementa o conteúdo para o qual p aponta. Como p está em n[1] (que vale 80), o valor passa a ser 81.*

O segundo printf imprime: Valor: 81.

Alternativa correta: d) Valor: 80 Valor: 81

3) Considere as variáveis declaradas no trecho de código a seguir. Faça um desenho ilustrando essas variáveis e seus conteúdos na situação apresentada e depois responda às questões formuladas.

```
char texto[25], palavra[15], descr[11], *p1, *p2;
strcpy(palavra, "xxxxxxx");
strcpy(texto, "Lingua de trapo");
scanf("%s", palavra + 3);
p1 = &texto[6];
p2 = &descr[0];
scanf("%s", p2); /* O usuário digita "goiaba" aqui */
strcpy(p1, " de trapo");
```

a) Considerando que no primeiro `scanf()` o usuário digitou "Banana", qual o conteúdo da variável `palavra` após essa entrada de dados? Qual seria o resultado da instrução `strlen(palavra)` nesse caso? E qual o resultado para `strlen(&palavra[3])`?

R.: Conteúdo de palavra: O vetor palavra iniciou com "xxxxxxx". O comando scanf("%s", palavra + 3) armazena a string a partir do quarto caractere (índice 3). A string "Banana" sobrescreve do índice 3 em diante. Assim, o conteúdo final armazenado na string será "xxxBana\0" (nota: o caractere nulo de terminação \0 é inserido automaticamente pelo scanf após o último caractere lido).

strlen(palavra): Conta os caracteres até encontrar o \0. Como as posições 0, 1 e 2 mantêm 'x', seguidas por "Bana", o comprimento total será 7.

strlen(&palavra[3]): Conta a partir do índice 3 (onde começa "Banana" abreviada por sobrescrita). O resultado será 4 (correspondente a "Bana").

b) Qual o conteúdo da variável `descr`?

R.: O usuário digitou "goiaba", que foi copiado para descr através de p2. Portanto, o conteúdo da variável será "goiaba" (seguido do caractere nulo \0).

c) Por que a chamada a `scanf()` não tem '`&`' quando o conteúdo digitado destina-se a uma *string*?

R.: Em C, o nome de um vetor (ou o ponteiro correspondente) já representa o endereço de memória do seu primeiro elemento. Como a função scanf() exige obrigatoriamente o endereço de memória da variável para poder alterar o seu valor diretamente, passar apenas o nome da string (que é um ponteiro para o início dela) já cumpre esse requisito, tornando redundante ou incorreto o uso do operador &.